

Lab Report No 5
Understanding and implementing K Mean
Clustering algorithm

Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

25th Mar, 2025

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
K-mean clustering: The K-Means Clustering Algorithm is used in this program to cluster customers based on their income and their spending score. The algorithm initializes centroids randomly, assigns them to the nearest, and updates repeatedly until convergence is reached. The optimal number of clusters is arrived at by using the Elbow Method which uses WCSS plots, while both unclustered and clustered data are visualized using scatter plots.
The code
<pre>import numpy as np import pandas as pd import matplotlib.pyplot as plt import random as rd # Load the dataset dataset = pd.read_csv('Mall_Customers.csv') # Select the relevant columns (Annual Income and Spending Score) X = dataset.iloc[:, [3, 4]].values # Number of training examples and features m = X.shape[0] # Number of data points n = X.shape[1] # Number of features # Number of iterations n_iter = 100 # Number of clusters K = 5 # Step 1: Randomly initialize the centroids Centroids = np.array([]).reshape(n, 0) for i in range(K): rand = rd.randint(0, m - 1) Centroids = np.c_[Centroids, X[rand]] # Step 2: K-means clustering algorithm for i in range(n_iter): # Step 2.a: Calculate the Euclidean distance for each point EuclidianDistance = np.array([]).reshape(m, 0) for k in range(K): tempDist = np.sum((X - Centroids[:, k]) ** 2, axis=1)</pre>

```

    EuclidianDistance = np.c_[EuclidianDistance, tempDist]
# Assign each point to the nearest cluster
C = np.argmin(EuclidianDistance, axis=1) + 1
# Step 2.b: Group points into clusters and update centroids
Y = {}
for k in range(K):
    Y[k + 1] = np.array([]).reshape(2, 0)
for i in range(m):
    Y[C[i]] = np.c_[Y[C[i]], X[i]]
for k in range(K):
    Y[k + 1] = Y[k + 1].T
for k in range(K):
    Centroids[:, k] = np.mean(Y[k + 1], axis=0)
Output = Y
# Visualize the unclustered data
plt.scatter(X[:, 0], X[:, 1], c='black', label='Unclustered Data')
plt.xlabel('Income')
plt.ylabel('Spending Score')
plt.legend()
plt.title('Plot of Data Points')
plt.show()
# Visualize the clustered data
color = ['red', 'blue', 'green', 'cyan', 'magenta']
labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5']
for k in range(K):
    plt.scatter(Output[k + 1][:, 0], Output[k + 1][:, 1], c=color[k], label=labels[k])
plt.scatter(Centroids[0, :], Centroids[1, :], s=300, c='yellow', label='Centroids')
plt.xlabel('Income')
plt.ylabel('Spending Score')
plt.legend()
plt.title('Clustered Data')
plt.show()
# Step 3: Determine the optimal number of clusters using the Elbow method
WCSS = [] # Within-Cluster Sum of Squares
for i in range(1, 11): # Try clusters from 1 to 10
    K = i
    Centroids = np.array([]).reshape(n, 0)
    for i in range(K):
        rand = rd.randint(0, m - 1)
        Centroids = np.c_[Centroids, X[rand]]

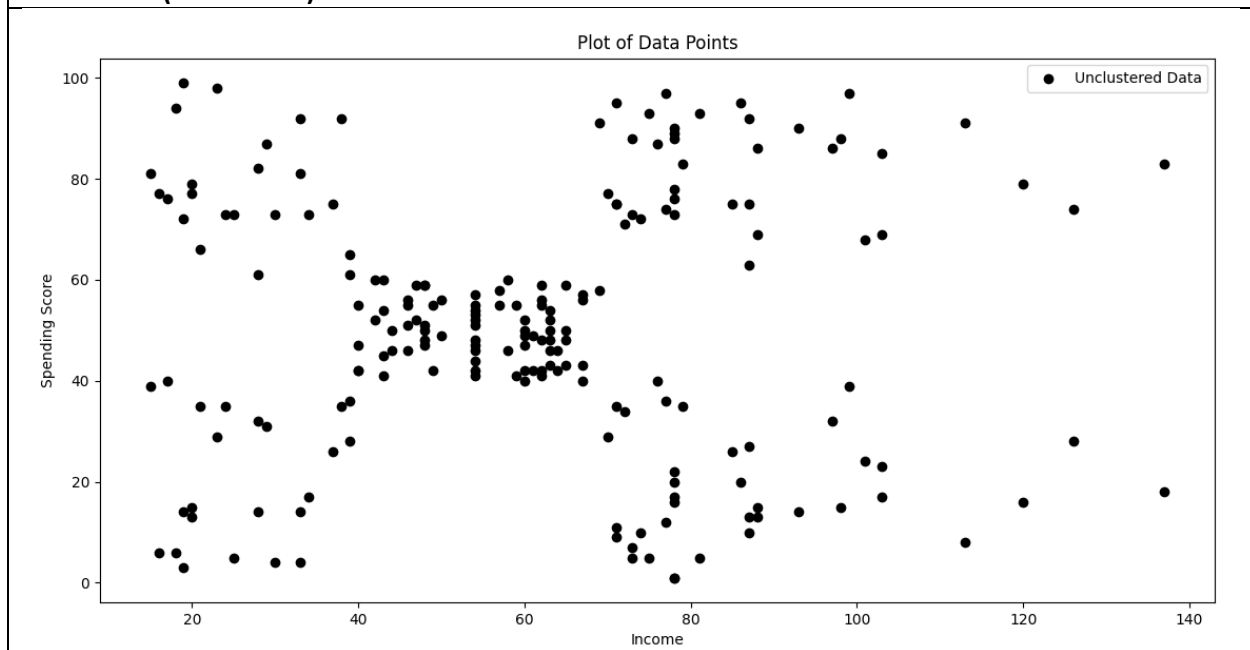
```

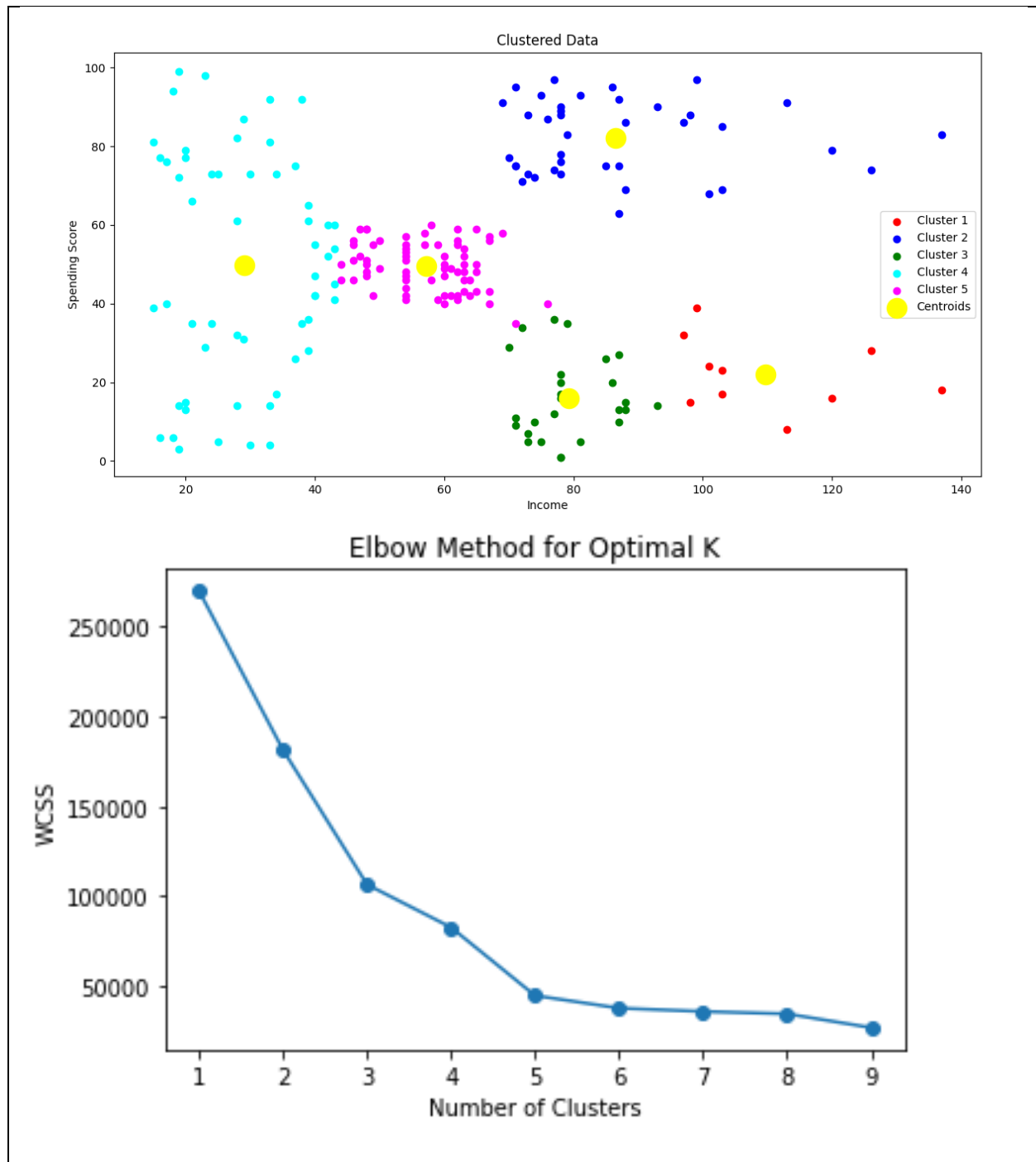
```

for i in range(n_iter):
    EuclidianDistance = np.array([]).reshape(m, 0)
    for k in range(K):
        tempDist = np.sum((X - Centroids[:, k]) ** 2, axis=1)
        EuclidianDistance = np.c_[EuclidianDistance, tempDist]
    C = np.argmin(EuclidianDistance, axis=1) + 1
    Y = {}
    for k in range(K):
        Y[k + 1] = np.array([]).reshape(2, 0)
    for i in range(m):
        Y[C[i]] = np.c_[Y[C[i]], X[i]]
    for k in range(K):
        Y[k + 1] = Y[k + 1].T
    for k in range(K):
        Centroids[:, k] = np.mean(Y[k + 1], axis=0)
# Calculate WCSS (Within-Cluster Sum of Squares)
WCSS.append(np.sum(np.min(EuclidianDistance, axis=1)))
# Plot the WCSS vs. number of clusters to find the "elbow"
plt.plot(range(1, 11), WCSS, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()

```

The results (Screenshot)





Lab Task No 1:

Solution:

Brief description (3-5 lines)

Mall Customer:

The mall customer theory in AI refers to a suite of applications employed to analyze shopper behavior using machine learning, computer vision, and data analytics. AI models are used to analyze data from sources such as CCTV, Wi-Fi tracking, and purchase history to infer patterns in foot traffic, customer preferences, and spending habits. Recommendation systems personalize the shopping experience, whereas predictive analytics help determine store placement and marketing strategies. Such insights, derived by AI, augment customer engagement, boost sales, and improve mall management efficiency.

The code

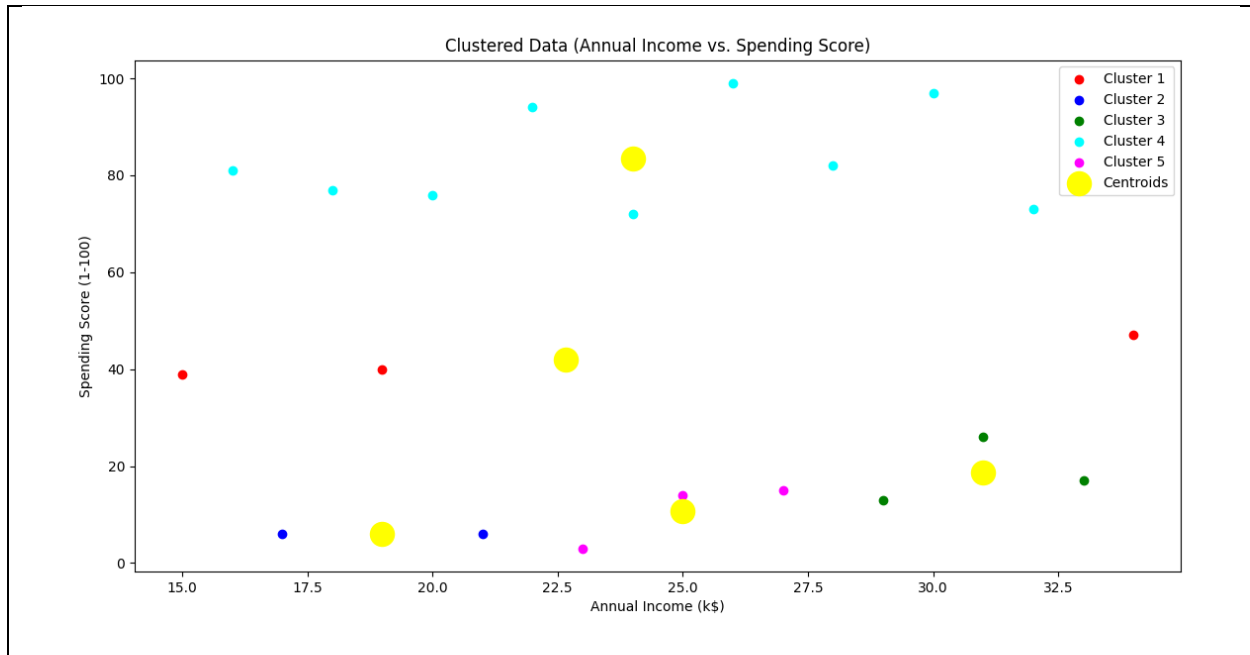
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random as rd
# Create dataset: Age, Annual Income (k$), and Spending Score (1-100)
data = {
    "CustomerID": np.arange(1, 21),
    "Age": [19, 21, 20, 23, 31, 22, 35, 23, 64, 30, 67, 35, 58, 24, 37, 22, 25, 40, 29, 50],
    "Annual Income (k$)": [15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34],
    "Spending Score (1-100)": [39, 81, 6, 77, 40, 76, 6, 94, 3, 72, 14, 99, 15, 82, 13, 97, 26, 73, 17, 47]
}
# Convert to DataFrame
df = pd.DataFrame(data)
# Save dataset as CSV file
df.to_csv("Mall_Customers.csv", index=False)
print("Dataset saved as 'Mall_Customers.csv'.")
# Load dataset
dataset = pd.read_csv("Mall_Customers.csv")
# Select features: Age, Annual Income, and Spending Score
X = dataset.iloc[:, [1, 2, 3]].values
# Number of clusters
K = 5
n_iter = 100
# Randomly initialize centroids
m, n = X.shape
Centroids = np.array([]).reshape(n, 0)
for i in range(K):
    rand = rd.randint(0, m - 1)
```

```

Centroids = np.c_[Centroids, X[rand]]
# K-means clustering algorithm
for i in range(n_iter):
    EuclidianDistance = np.array([]).reshape(m, 0)
    for k in range(K):
        tempDist = np.sum((X - Centroids[:, k]) ** 2, axis=1)
        EuclidianDistance = np.c_[EuclidianDistance, tempDist]
    # Assign clusters
    C = np.argmin(EuclidianDistance, axis=1) + 1
    # Update centroids
    Y = {k + 1: np.array([]).reshape(n, 0) for k in range(K)}
    for i in range(m):
        Y[C[i]] = np.c_[Y[C[i]], X[i]]
    for k in range(K):
        Y[k + 1] = Y[k + 1].T
    for k in range(K):
        Centroids[:, k] = np.mean(Y[k + 1], axis=0)
# Visualizing clustered data (Using only 2D: Annual Income vs. Spending Score)
color = ['red', 'blue', 'green', 'cyan', 'magenta']
labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5']
for k in range(K):
    plt.scatter(Y[k + 1][:, 1], Y[k + 1][:, 2], c=color[k], label=labels[k])
# Plot centroids
plt.scatter(Centroids[1, :], Centroids[2, :], s=300, c='yellow', label='Centroids')
# Labels and title
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.title('Clustered Data (Annual Income vs. Spending Score)')
plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, K-Means clustering is an unsupervised, powerful algorithm that partitions data into K clusters based on feature similarity. It is computationally efficient and applicable in various domains like customer segmentation, image compression, and pattern recognition. However, the algorithm's performance is sensitive to initial centroids, cluster selection, and spherical shape assumptions. Practitioners need to preprocess data, perform tests with different K values, and use it in combination with other techniques for optimal results.